COURSE PROJECT REPORT · WEB APPLICATION DEVELOPMENT

Placement Portal

A Role-Based Web Application for Campus Placement Management

Flask 3.0 | SQLite + SQLAlchemy | Bootstrap 5 | Flask-Login | Jinja2

SUBMITTED BY
[AMAN CHAWALA/ 23F3001753]

COURSE
Mordern Application Development t1

VIDEO LINK
[https://drive.google.com/file/d/1QIE5QxHFHksZeBuWReh8JnKT0JO0l6gH/view?usp=sharing]

1 PROJECT OVERVIEW

The Placement Portal is a full-stack web application that digitises the campus placement process for a university. It provides a unified platform for three distinct user roles — **Admin**, **Company**, and **Student** — each with a protected dashboard and role-specific operations.

Built with **Flask (Python)** as the backend, **Jinja2** for server-side templating, **Bootstrap 5** for the frontend, and **SQLite + SQLAlchemy ORM** for the database. The application runs entirely locally with a single `python app.py` command and requires no manual database setup.

2 OBJECTIVES

Build a role-based placement management system for a university environment Automate the workflow of company registration, drive creation, student applications, and admin approvals

Enforce data integrity (no duplicate applications) at both application and database levels

Keep the project clean, modular, and viva-explainable with no JavaScript-based logic

Enable zero-setup deployment — DB and default admin auto-created on first run

3 TECHNOLOGY STACK

Layer	Technology	Version / Notes
Backend Framework	Flask (Python)	3.0.0

Layer	Technology	Version / Notes
Templating Engine	Jinja2	Built into Flask
Frontend UI	Bootstrap 5 + Bootstrap Icons	5.3.2 (CDN)
Database	SQLite	File-based, no server required
ORM	Flask-SQLAlchemy	3.1.1
Authentication	Flask-Login	0.6.3
Password Hashing	Werkzeug Security	3.0.1
File Uploads	Werkzeug secure_filename	Built into Werkzeug
Language	Python 3.x	—

4 PROJECT FOLDER STRUCTURE

```
placement_portal/
├── app.py                  ← Main application (routes + logic + DB init)
├── config.py               ← Centralised configuration settings
├── models.py               ← SQLAlchemy database models (5 models)
├── requirements.txt        ← Python dependencies (pinned versions)
├── static/
│   ├── css/
│   │   └── style.css       ← Custom styles (extends Bootstrap)
│   └── uploads/            ← Stores uploaded student resumes
│       └── .gitkeep
└── templates/
    ├── base.html            ← Master template (navbar, flash, footer)
    ├── login.html           ← Unified login page (all roles)
    ├── register_student.html
    ├── register_company.html
    ├── admin_dashboard.html ← Admin control panel
    ├── company_dashboard.html
    ├── student_dashboard.html
    ├── drives.html          ← Drive create / edit form (shared)
    └── applications.html   ← Application list (student + company view)
```

5 DATABASE DESIGN

The application uses **5 SQLAlchemy models** stored in a single SQLite file (`placement_portal.db`):

5.1 User (Table: `users`)

Central user table shared across all three roles. Flask-Login uses this model for session management.

Column	Type	Description
id	Integer (PK)	Auto-incremented primary key
name	String(100)	Full name of user
email	String(120) UNIQUE	Login email — must be unique
password	String(200)	Bcrypt-hashed password — never plain text
role	String(20)	admin / company / student
is_active	Boolean	False = blacklisted, Flask-Login blocks login
is_approved	Boolean	True = admin has approved the account

5.2 CompanyProfile (Table: `company_profiles`)

Column	Type	Description
id	Integer (PK)	Primary key
user_id	FK → users.id	Linked user account (one-to-one)
company_name	String(200)	Company name
hr_contact	String(100)	HR contact number or email
website	String(200)	Company website URL
approval_status	String(20)	Pending / Approved / Rejected

5.3 StudentProfile (Table: `student_profiles`)

Column	Type	Description
id	Integer (PK)	Primary key
user_id	FK → users.id	Linked user account (one-to-one)
course	String(100)	Degree / programme name
cgpa	Float	Academic score (validated: 0.0 – 10.0)
contact_number	String(15)	Mobile number
resume_filename	String(200)	Uploaded resume filename (not full path)

5.4 PlacementDrive (Table: `placement_drives`)

Column	Type	Description

id	Integer (PK)	Primary key

Column	Type	Description
company_id	FK → company_profiles.id	Drive owner
job_title	String(200)	Position name
job_description	Text	Full job description
eligibility	String(300)	Eligibility criteria
deadline	Date	Application closing date
status	String(20)	Pending → Approved → Closed / Rejected
created_at	DateTime	Auto-set on creation

5.5 Application (Table `applications`)

Column	Type	Description
id	Integer (PK)	Primary key
student_id	FK → student_profiles.id	Applicant
drive_id	FK → placement_drives.id	Target drive
application_date	DateTime	Auto-set timestamp of application
status	String(20)	Applied / Shortlisted / Selected / Rejected

⚠ **Unique Constraint:** `(student_id, drive_id)` — enforced at the database level. A student physically cannot apply twice to the same drive, even if the application-level check is bypassed.

6 ENTITY RELATIONSHIP DIAGRAM

```
users (1) ──────────── (1) company_profiles (1) ─── (many) placement_drives
│ users (1) ──────────── (1) student_profiles  (1) ─── (many) applications
│                                              placement_drives (1) ──────┘

Relationships:
  User          ──(1:1)──  CompanyProfile    [via user_id FK]
User          ──(1:1)──  StudentProfile    [via user_id FK]
  CompanyProfile ──(1:N)── PlacementDrive    [via company_id FK]
  StudentProfile ──(1:N)── Application       [via student_id FK]
  PlacementDrive ──(1:N)── Application       [via drive_id FK]
```

7 APPLICATION WORKFLOW

7.1 Authentication Flow

All three roles share a single `/login` page

Login validates: correct password → account is active → (for company) admin-approved

On success, Flask-Login stores the user ID in a signed session cookie

`@login_required` decorator protects all dashboard routes

Each dashboard additionally checks `current_user.role` to prevent cross-role access

7.2 Company Workflow

```
Register → [Admin Approves] → Login → Post Drive → [Admin Approves Drive]
→ Students Apply → View Applicants → Update Application Status → Close Drive
```

7.3 Student Workflow

```
Register → Login → View Approved Drives → Apply (one-click POST form)
→ Track Application Status → Upload Resume via Profile Page
```

7.4 Admin Workflow

```
Auto-login (admin@portal.com / admin123) → View Platform Stats
→ Approve / Reject Companies → Approve / Reject Drives
→ Search Students / Companies → Blacklist / Reactivate Users
```

8 KEY FEATURES

#	Feature	Implementation Detail
1	Role-Based Access Control	Every route: @login_required + current_user.role check + abort(403) for API endpoints
2	Auto Admin Initialisation	create_default_admin() called on startup inside app_context — skips if already exists
3	Duplicate Application Prevention	UniqueConstraint('student_id','drive_id') in DB + query check in route before insert
4	Company Approval Workflow	Companies register as Pending; admin approves; login block enforced at login route
5	Drive Re-approval on Edit	Editing a drive resets status to Pending — must be re-approved before students can see it

#	Feature	Implementation Detail
6	Resume Upload	PDF/DOC/DOCX, max 5 MB, secure_filename() sanitises path, stored in static/uploads/
7	Search Functionality	Admin searches students (name/email) and companies via SQL ilike — case-insensitive
8	User Blacklisting	Toggle is_active — Flask-Login auto-blocks blacklisted users on every request
9	Deadline Enforcement	apply_drive() checks drive.deadline < date.today() — rejects late applications
10	Applicant Status Updates	Company updates: Applied → Shortlisted → Selected / Rejected via POST form

9 SECURITY MEASURES

Threat	Measure	Flask Component
Plain-text passwords	Bcrypt hash via generate_password_hash()	Werkzeug Security
Session hijacking	Signed session cookie with SECRET_KEY	Flask Sessions
Malicious file uploads	secure_filename() + extension whitelist + 5 MB limit	Werkzeug Utils + Config
Cross-role access	Role check on every route + abort(403)	Custom Python guard
Unauthorised drive edit	drive.company_id == company.id check	Ownership verification
SQL Injection	ORM parameterised queries — no raw SQL	Flask-SQLAlchemy
XSS Attacks	Jinja2 auto-escapes all template variables	Jinja2 Default
Admin self-blacklist	user_id == current_user.id guard in blacklist route	Custom Python guard

10 FILE DESCRIPTIONS

File	Role / Purpose
app.py	Flask app setup, all 18+ route handlers, login/logout logic, DB initialisation on startup
config.py	Config class — SECRET_KEY, SQLite DB URI, upload folder path, allowed extensions

models.py	All 5 SQLAlchemy models with relationships, constraints, and cascade rules
requirements.txt	Pinned dependencies (Flask, SQLAlchemy, Flask-Login, Werkzeug) for reproducible installs

File	Role / Purpose
base.html	Master Jinja2 template — navbar (role-aware), flash messages, footer, Bootstrap imports
login.html	Single login page used by all three roles; shows role-specific error messages
register_student.html	Student registration — validates CGPA (0.0–10.0), creates User + StudentProfile
register_company.html	Company registration — creates User + CompanyProfile with Pending status
admin_dashboard.html	Stats cards, company/student management tables, pending drives list, search forms
company_dashboard.html	Drive list with applicant counts; links to edit, close, and view applicants
student_dashboard.html	Approved drives (sorted by deadline), applied_drive_ids set (hides apply button if applied)
drives.html	Reusable form — handles both create (action=create) and edit (action=edit) via Jinja2 conditionals
applications.html	Dual-purpose — company view shows all applicants; student view shows own history with status
style.css	Custom CSS on top of Bootstrap 5 — card hovers, badge colours, table enhancements

11 HOW TO RUN

```
# Step 1: Navigate to the project folder
cd placement_portal

# Step 2: Create a virtual environment
python -m venv venv

# Step 3: Activate the virtual environment
venv\Scripts\activate          # Windows
# source venv/bin/activate       # Mac / Linux

# Step 4: Install all dependencies
pip install -r requirements.txt

# Step 5: Start the application
python app.py

# Step 6: Open in browser

# http://127.0.0.1:5000

# ────────────────────────────────────────
#  Default Admin Credentials
#  Email    : admin@portal.com
#  Password: admin123
# ────────────────────────────────────────
```

The SQLite database (`placement_portal.db`) and the default admin account are **automatically created on the first run** No manual database setup or SQL commands are required.

12 LIMITATIONS AND FUTURE SCOPE

Current Limitation	Possible Enhancement
Local SQLite only — not suitable for concurrent users	Migrate to PostgreSQL / MySQL for production deployment
No email notifications for approvals or applications	Integrate Flask-Mail for automated email alerts
No pagination on dashboard tables	Implement SQLAlchemy paginate() for large datasets
Basic search (name/email only)	Advanced filters — CGPA range, course, application date, drive status
Manual shortlisting by company	Auto-shortlisting based on configurable CGPA threshold per drive

No analytics charts	Add Chart.js placement success graphs on admin dashboard
No password reset flow	Email-based password reset using Flask-Mail + timelimited tokens

13 CONCLUSION

The Placement Portal successfully implements a complete, role-based web application for managing campus placements. The project demonstrates practical use of Flask route handling, SQLAlchemy ORM with relational models, Flask-Login session management, Jinja2 server-side templating, and a Bootstrap-based responsive UI.

The application is modular, well-commented, and requires zero manual setup. All core logic is implemented in Python/Flask — no JavaScript-based functionality — making the codebase clean, readable, and fully explainable during a viva examination.

Video Presentation Link: [https://drive.google.com/file/d/1QIE5QxHFHksZeBuWReh8JnKT0JO0l6gH/view?usp=sharing]